Clean Code

Whether you are a student, professional, or even a recreational programmer, clean code should be a way of life. But what is clean code, and why should we care?

CONTENTS

Clean Code Matters • Read These First • Comments • Testing • Formatting and Naming • DRY Code • Conceptual Organization • Efficiency • Safety and Security • Compactness • Be Reasonable and Pragmatic

Clean Code Matters

Coding is a form of writing. When you code, you are expressing yourself. Code cleanly, to show pride in your work, and, importantly, that you care about (1) yourself, (2) the teammates that will read your code, and (3) the users that are trusting in the correctness of your software.

If your code is not clean, it is error-prone, hard to reason about, and hard to test.

Cleanliness makes correctness and efficiency easier to achieve.

Read These First

For a quick introduction to the topics and techniques addressed by the umbrella term *Clean Code*, begin by reading this summary of the Clean Code principles from Robert C. Martin's *Clean Code* book.

Next, read this more detailed article, explaining some of the principles. (It's okay if you don't understand every single principle for now; many are Java-specific since this book dates from way back in 2008 when Java was king, but most are very general.)

Finally, browse the "Smells and Heuristics" section of Martin's book. It's good to know what kinds of coding constructs to avoid as well as what kinds to write.

> ***Wait what's all this about a single book by a single individual?***
>
> *Well, Martin's book is called "Clean Code" after all. And a lot of what he says in the book is reasonable. Some of it my even be called timeless. Some of it is common sense. But the book has flaws and it is NOT gospel.*

Next, and this is important too, read this negative review of the Clean Code book to add some balance!

Next, free yourself from thinking Martin's book is the last word. If you can, read The Art of Readable Code instead (it's good).

Now that you have a sense of what the term "Clean Code" refers to, and you know that people can disagree on some of the details of what is and what is not clean code, and you know not to adopt certain practices just because some individual carved them in stone, I've added my own short list of things in the world of clean code that I find important.

Comments

"Nothing can be quite so helpful as a well-placed comment. ... Nothing can be quite so damaging as an old crufty comment that propagates lies and misinformation."—Robert C. Martin

- ✅ Prefer code that says what it means to code that needs comments.
- ✅ Use comments to clarify things that cannot be said in the code, such as the *intent* (rather than the inner workings) of a non-trivial, dense, or cryptic block of code.
- ✅ A comment at the very top of a file that tells the reader the purpose of the file *may* (but not always) be useful.
- ✅ Where required, licenses and copyright notices can appear in comments.
- ✅ You can use a comment to say who helped you with some code, whose bright idea led to the code, or a hyperlink to the source of the idea or algorithm behind the code. Always be a good citizen and give attribution!

- ✅ Comments should always be at a level of abstraction above the code; they must never “repeat the code.”
- ✅ Think twice before commenting a function or method. Maybe you should pick a really good descriptive name for the function instead. Still thinking of that comment? Try a better name.

Testing

Not writing unit tests is not an option.

- ✅ Write unit tests properly. Printing is for debugging, not testing.
- ✅ Cover all boundary cases and potentially troublesome cases: zero, negative, empty collection, full collection, huge numbers, etc.
- ✅ Make sure your tests are fully independent from each other: no test should depend on a state change made by another test.
- ✅ Separate unit tests from integration tests.
- ✅ Unit tests should never hit an external system (database, network, etc.)
- ✅ Each unit test should run very fast.

Formatting and Naming

Sloppy formatting can be construed as being lazy or not taking pride in yourself or your work. It reflects badly on your competence and potentially on your worth as a prospective employee. Your code should, in addition to being correct, look beautiful. It should read, in the words of Grady Booch, “like well-written prose.”

- ✅ Be always and absolutely consistent with indentation, spacing, and capitalization.
- ✅ Don’t mix camel case and underscores within the same identifier class; again, consistency is paramount.

- ✅ It is best to always save code with spaces, never tabs, because when someone else reads your code with tabs it will often look horrifying. Tabs alone often translate into 8-character indentations, and a mix of tabs and spaces can look like lines of code were randomly dropped on the floor.
- ✅ Names should be pronounceable. Don't use partial abbreviations like *pnt* for point, or *nbr* for number, or *WndMgr* for WindowManager.
- ✅ Parts of speech matter. Be very careful to use nouns for things and properties, verbs for actions, and so on.
- ✅ Only abbreviate if you have a loop variable, a parameter or local variable with tiny scope or a well-accepted and understood acronym.

Note that the use of a linter or code prettier can check for these items and more, and generally automatically fix any problems. Once you have learned the basics of programming and have gotten familiar with editing and running programs, use these tools.

- ✅ Use an auto formatter (e.g., Prettier).
- ✅ Use a linter.

DRY Code

So many bugs have been traced to "I remembered to change it here but not there."

- ✅ **One fact in one place**: a simple change should not require you to make edits in multiple places in the code.
- ✅ Don't hardcode data that is supposed to be algorithmically generated.
- ✅ Don't use hardcoded literals, sometimes called magic numbers, except for things like 0, 1, and sometimes 2.
- ✅ **However, you do not have to be "as DRY" in unit tests**! Being DAMP is sometimes better when testing.

> ***You have some flexibility with DRYness***

In rare situations it is possible to be "too" dry. Ask an expert when unsure.

Conceptual Organization

Large systems can be more understandable if they are constructed, layered, and wired together cleanly.

- ✅ Ensure that each of the "subsystems" or "roles" or "activities" of a system are separated.
- ✅ Avoid making data that is "too global", strive to localize declarations.
- ✅ Try not to mix user input with business logic with output.
- ✅ Don't mix statements of different complexities in the same function. For example, it's generally best to avoid functions that have some statements that are high-level function calls together with other statements with intricate logic and low-level mathematical gymnastics.
- ✅ Modules (or classes/object) should have a single, identifiable responsibility. They should be internally cohesive, but have very small interfaces and dependencies on other modules.
- ✅ Make dependencies general: for example don't write a component that depends on a Postgres driver; make it dependent on an arbitrary driver and pass the specific driver in as a parameter. (This is known as **dependency injection** and it's generally a very good thing.)

Efficiency

Sometimes a readability versus performance tradeoff pops up, but quite often the most readable code actually *is* the most efficient. And sometimes, your code may be prone to efficiency attacks, so watch out.

- ✅ Don't call functions (like `strlen`, say) over and over again if they will always be returning the same result. If you need the result of a long-running function call more than once, you probably want to cache it.
- ✅ Watch out for functions that would take forever if passed huge numbers.

Safety and Security

This section is a little different, being not about style but rather about responsibilities.

- ✅ Make sure your code is resilient for all inputs. Don't let your application crash ungracefully. Detect malicious inputs. Never trust anyone.
- ✅ Don't write code that accumulates roundoff errors.
- ✅ Always check for `null` before dereferencing. look up a property from through a reference (generally through `.` or `->`), ask yourself whether there is any way the pointer can be null. If your language has a `?.` operator, use it!
- ✅ In C, Match your `mallocs` and `frees`.
- ✅ In any unmanaged language, When writing a opaque data structure module, remember to include a `destroy` operation.
- ✅ In C, Remember to leave space for the zero byte at the end of a string.
- ✅ In C, Never do a `strcpy` of a passed in string into a fixed-size local string variable.
- ✅ In C, Never use `strcpy`.
- ✅ In general, never assume your buffer is big enough.
- ✅ Familiarize yourself with the CERT C Coding Standard
- ✅ Familiarize yourself with the CERT C++ Coding Standard
- ✅ Familiarize yourself with the CERT Java Coding Standard

Compactness

Often, compact code is more readable than drawn-out code.

- ✅ Don't say `if found==true`, instead say `if found`

- ✅ Don't say `if index>0 return true else return false`, instead say `return index>0`
- ✅ Capture information in data where possible.
- ✅ Use lookups instead of conditional logic where appropriate.

Be Reasonable and Pragmatic

You need to get along with others, don't be mean and dogmatic.

- ✅ Remember (for the most part) these are guidelines are not laws.
- ✅ Beware of debates about whether someone's style is *always* better than other.
- ✅ There is room for nuance and pragmatism.
- ✅ You can always follow existing published style guides, such as these by Google.